

Applicazione ha 3 layer

Premessa.....	1
Premessa.....	2
Analisi del problema.....	3
Caso d'uso.....	3
Progettazione.....	4
Presentation.....	5
View.....	5
Controller.....	5
Model.....	6
Progettazione interfaccia utente.....	6
Layer Business.....	7
Layer Business.....	7
Componenti.....	7
Pattern utilizzati.....	7
DAO Pattern.....	7
Proxy Pattern (EJB).....	8
Oggetti.....	8
Diagrammi.....	8
Layer Data.....	10
Conclusione.....	10

Premessa

Premessa

La seguente applicazione è un gestionale per la Segreteria Universitaria dedicato alla gestione degli studenti.

L'applicazione utilizza un'architettura **a 3 tier** ed è progettata per essere eseguita su un server interno con:

- Application Server **GlassFish**
- Database **MySQL**

Gli utenti accedono tramite browser, senza installare software locale.

Il progetto avrà il seguente Gant:

		Nome	Durata	Data di Avvio	Data di chiusura	Predecessori	Nome risorsa
1		Progetto Segreteria Università	7 giorni	21/01/26, 08:00	29/01/26, 17:00		Andre Scarpati ProjectMan...
2		Elis-off	1 giorno	21/01/26, 08:00	21/01/26, 17:00		
3		avvio progetto	1 giorno	21/01/26, 08:00	21/01/26, 17:00		
4		architettura 3layer	1 giorno	21/01/26, 08:00	21/01/26, 17:00		
5		Ambiente Glassfish +mysql	1 giorno	21/01/26, 08:00	21/01/26, 17:00		
6		definizione dei moduli	1 giorno	21/01/26, 08:00	21/01/26, 17:00		
7		Analisi	1 giorno	22/01/26, 08:00	22/01/26, 17:00	2	
8		Analisi requisiti	1 giorno	22/01/26, 08:00	22/01/26, 17:00		
9		Analisi casi d'uso	1 giorno	22/01/26, 08:00	22/01/26, 17:00		
10		diagrammi delle sequenze	1 giorno	22/01/26, 08:00	22/01/26, 17:00		
11		Progettazione	2 giorni	23/01/26, 08:00	26/01/26, 17:00	7	
12		Presentation	1 giorno	23/01/26, 08:00	23/01/26, 17:00		
13		Storyboard interfacce(home,ricerca,inserimento)	1 giorno	23/01/26, 08:00	23/01/26, 17:00		
14		diagramma delle componenti(servlet,jsp,pojo)	1 giorno	23/01/26, 08:00	23/01/26, 17:00		
15		Mappa sito	1 giorno	23/01/26, 08:00	23/01/26, 17:00		
16		Pattern mvc e tecnologia applicata view(jsp),servlet (controller),bootstrapp (ui)	1 giorno	23/01/26, 08:00	23/01/26, 17:00		
17		Pattern navigazione(menu e back navigator)	1 giorno	23/01/26, 08:00	23/01/26, 17:00		
18		Business Layer(UML,classi,stat)	1 giorno	26/01/26, 08:00	26/01/26, 17:00	12	
19		pattern Singleton e dao	1 giorno	26/01/26, 08:00	26/01/26, 17:00		
20		java bean pojo	1 giorno	26/01/26, 08:00	26/01/26, 17:00		
21		relazioni tra le classi	1 giorno	26/01/26, 08:00	26/01/26, 17:00		
22		Interfaccia aziendale	1 giorno	26/01/26, 08:00	26/01/26, 17:00		
23		Data Layer(e-r,logico,fisico)	1 giorno	26/01/26, 08:00	26/01/26, 17:00	12	
24		modello E-R,logico con phpmyadmin design mysql	1 giorno	26/01/26, 08:00	26/01/26, 17:00		
25		Struttura del package del programma	1 giorno	23/01/26, 08:00	23/01/26, 17:00		
26		Implementazione fisica	1 giorno	27/01/26, 08:00	27/01/26, 17:00	23;11;18;25	
27		JSP HOME	1 giorno	27/01/26, 08:00	27/01/26, 17:00		
28		JSP Inserimento	1 giorno	27/01/26, 08:00	27/01/26, 17:00		
29		servlet cotrollor	1 giorno	27/01/26, 08:00	27/01/26, 17:00		
30		classi java bean pojo e service e dao	1 giorno	27/01/26, 08:00	27/01/26, 17:00		
31		Script,sql generato da phpmyadmin /mysql Design)	1 giorno	27/01/26, 08:00	27/01/26, 17:00		
32		test drive e stubs	1 giorno	27/01/26, 08:00	27/01/26, 17:00		
33		Test finali	1 giorno	28/01/26, 08:00	28/01/26, 17:00	26	
34		Inserimento con h2 o test.mysql	1 giorno	28/01/26, 08:00	28/01/26, 17:00		
35		ricerca con h2 o test.mysql	1 giorno	28/01/26, 08:00	28/01/26, 17:00		
36		integrazione test specifici db	1 giorno	28/01/26, 08:00	28/01/26, 17:00		
37		Consegna	1 giorno	29/01/26, 08:00	29/01/26, 17:00	33	
38		Pacchetto finale	1 giorno	29/01/26, 08:00	29/01/26, 17:00		
39		verifica funzionamento	1 giorno	29/01/26, 08:00	29/01/26, 17:00		
40		Organizzazione file	1 giorno	29/01/26, 08:00	29/01/26, 17:00		
41		chiusura progetto	1 giorno	29/01/26, 08:00	29/01/26, 17:00		

Analisi del problema

La segreteria deve gestire gli studenti tramite:

- Registrazione di nuovi studenti
- Ricerca degli studenti già presenti

Il sistema deve essere accessibile da più postazioni interne tramite pagine web.

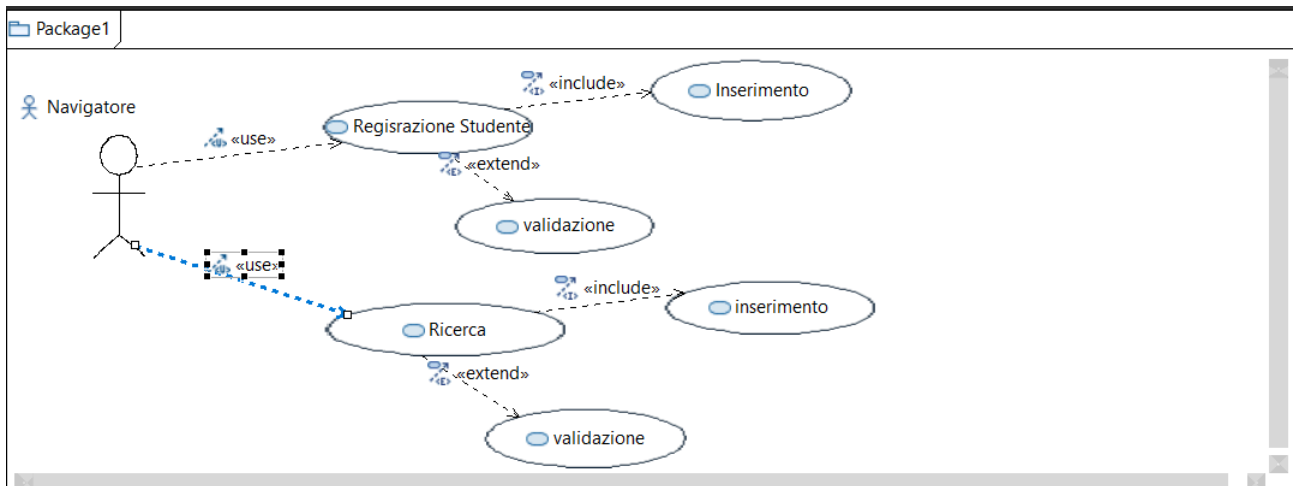
L'infrastruttura prevede:

- Server GlassFish per l'applicazione
- Server MySQL per la persistenza dei dati

Caso d'uso

L'utente:

1. Accede alla home
2. Seleziona “Ricerca” o “Inserimento”
3. Compila il form
4. Il sistema interroga il Business Layer
5. Il Business Layer interagisce con il Data Layer
6. Viene restituito l’esito

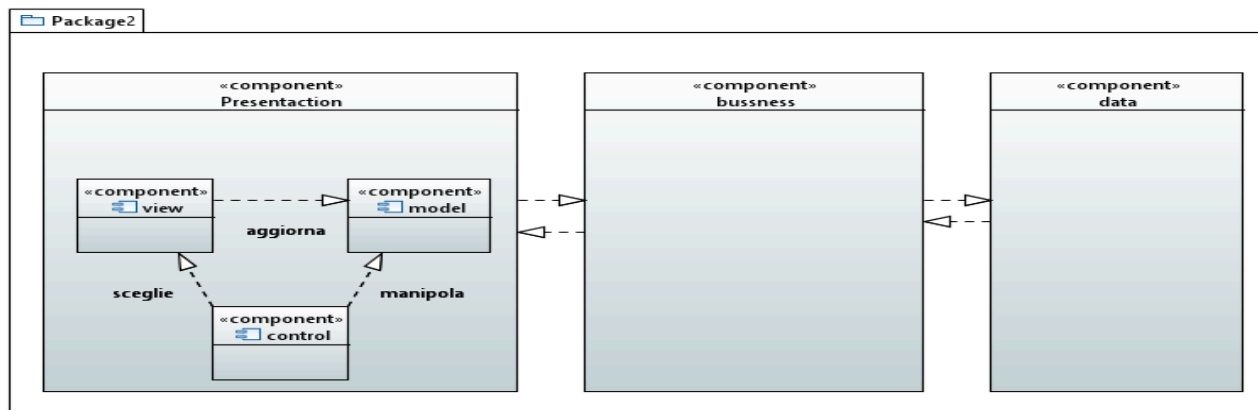


Progettazione

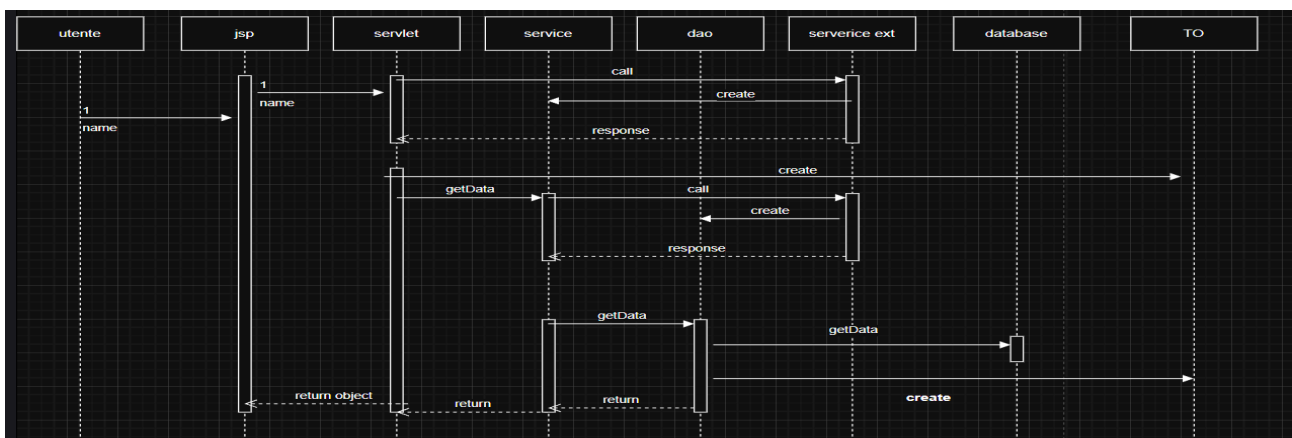
L'applicazione è sviluppata con architettura **a tre layer**:

- **Presentation Layer**
- **Business Layer**
- **Data Layer**

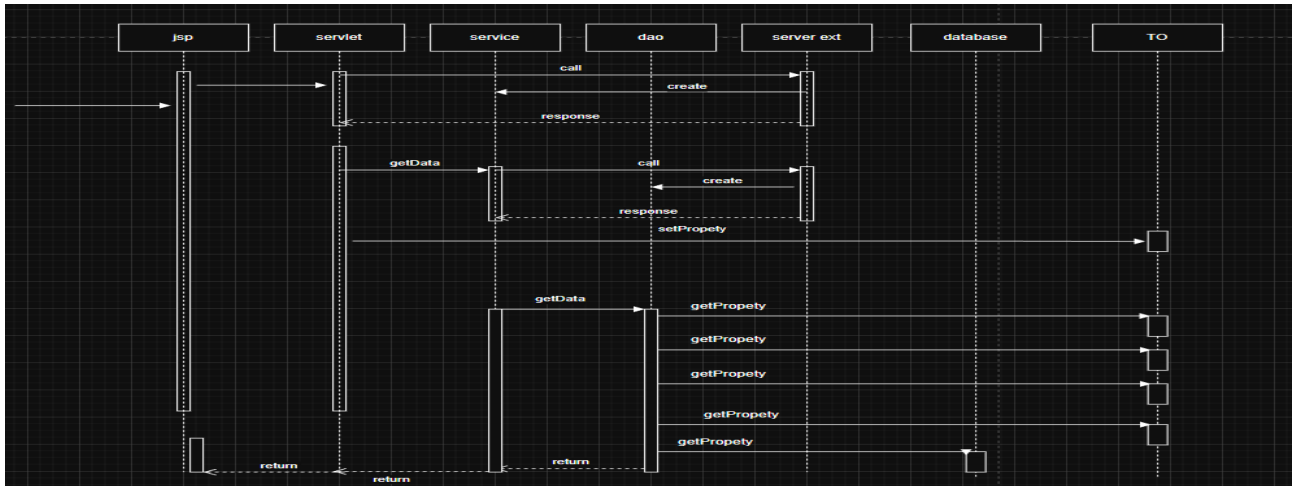
Sono previsti:



- Diagramma delle sequenze per la ricerca



- Diagramma degli stati per l'inserimento



Presentation

Lo strato di presentazione utilizza il pattern **MVC (Model–View–Controller)**.

View

Realizzata con:

- JSP
- Bootstrap CSS

La view è composta da 3 moduli:

1. **Home** con pattern *menu*
 2. **Modulo Ricerca Studente**
 3. **Modulo Inserimento Studente**
2. Le JSP vengono eseguite su GlassFish e generano HTML dinamico.

Bootstrap adatta la pagina ai vari dispositivi.

Controller

Realizzato con **Servlet Java**, che:

- riceve le richieste
- valida i dati
- invoca il Business Layer

Model

Realizzato con **JavaBean POJO**, che rappresentano gli studenti.

Progettazione interfaccia utente

Storyboard dei moduli:

- **Home** con pattern *web menu*

Esempio d'uso di *JavaBean*

[visualizzaStudente](#)[inserisciStudente](#)

- Modulo ricerca

Esempio d'uso di *JavaBean*

Cerca il cognome nel database

Cognome:	bcerca
-------------------------------	---------------------------------------

Nessun risultato trovato per il cognome inserito.

[Indietro](#)

\

- Modulo Inserimento

inserimento dati

Matricola	nome	cognome	datanascita	dataiscr
inserisci				

Studente inserito n: 0

[Indietro](#)

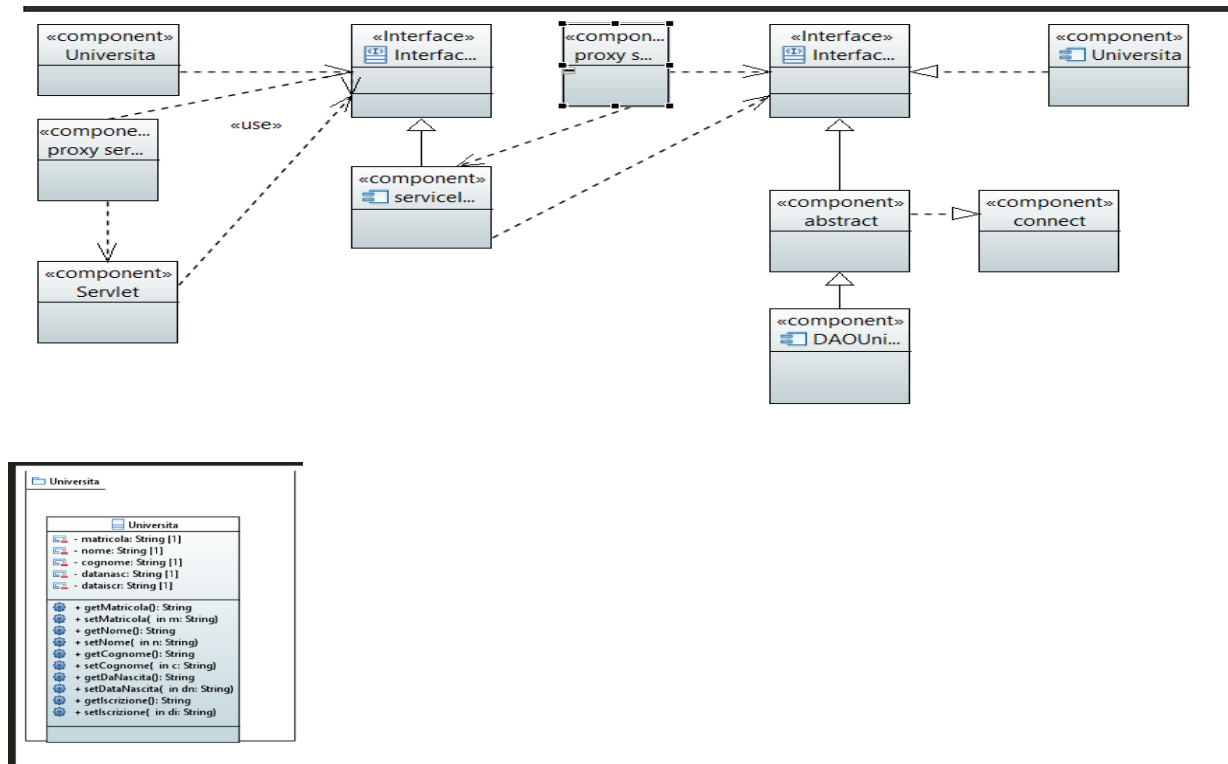
Nei moduli operativi è presente un pattern *web back* per tornare alla home

Layer Business

Layer Business

Ogni richiesta HTTP viene gestita dal server tramite un thread del thread pool. La Servlet viene eseguita in quel thread e invoca un EJB @Stateless, che non conserva stato tra le richieste, lo fa tramite file

In questo layer risiede la logica applicativa.



Componenti

- Interfaccia **Service** (@Local)
- Implementazione **Service** (@Stateless)
- **DAO** con struttura gerarchica
- **POJO** per i dati

Pattern utilizzati

DAO Pattern

Serve a:

- incapsulare la logica JDBC
- separare la logica DB dai POJO

Struttura:

- **AbstractDAO**
 - contiene il metodo **connect**
 - implementa un **Singleton** per la connessione
 - utilizza **DriverManager** con JDBC
- DAO concreti
 - implementano le operazioni CRUD

Proxy Pattern (EJB)

L'application server gestisce l'istanziamento:

- Nella Servlet → **@EJB** su attributo **serv** ce un associazione di tipo aggregazione tra servlet e il service
- Nel Service → **@Stateless** e **@EJB** sul DAO ce un associazione di tipo aggregazione tra il service e il dao
- Nel **Flusso**

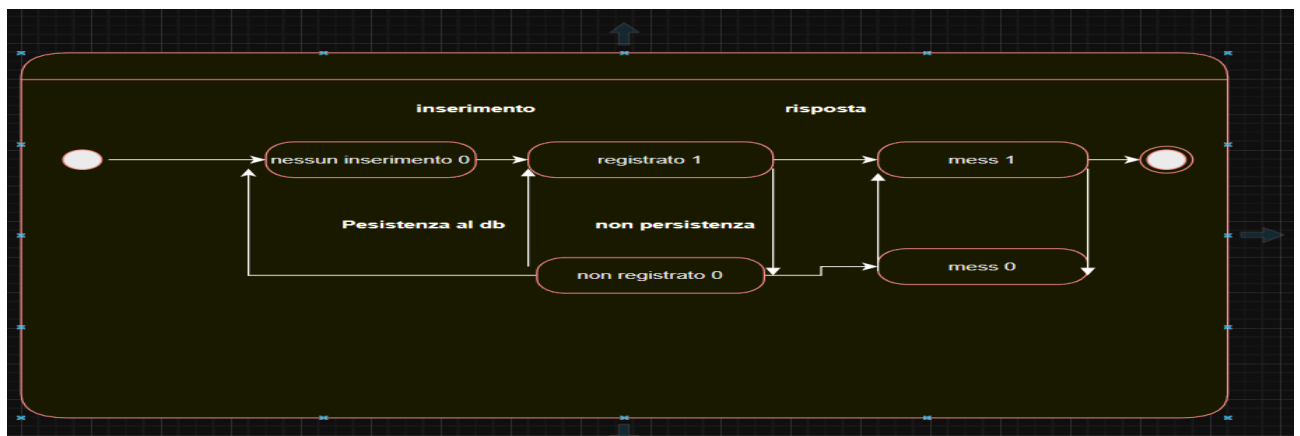
Servlet → Service → DAO → JDBC → MySQL

Oggetti

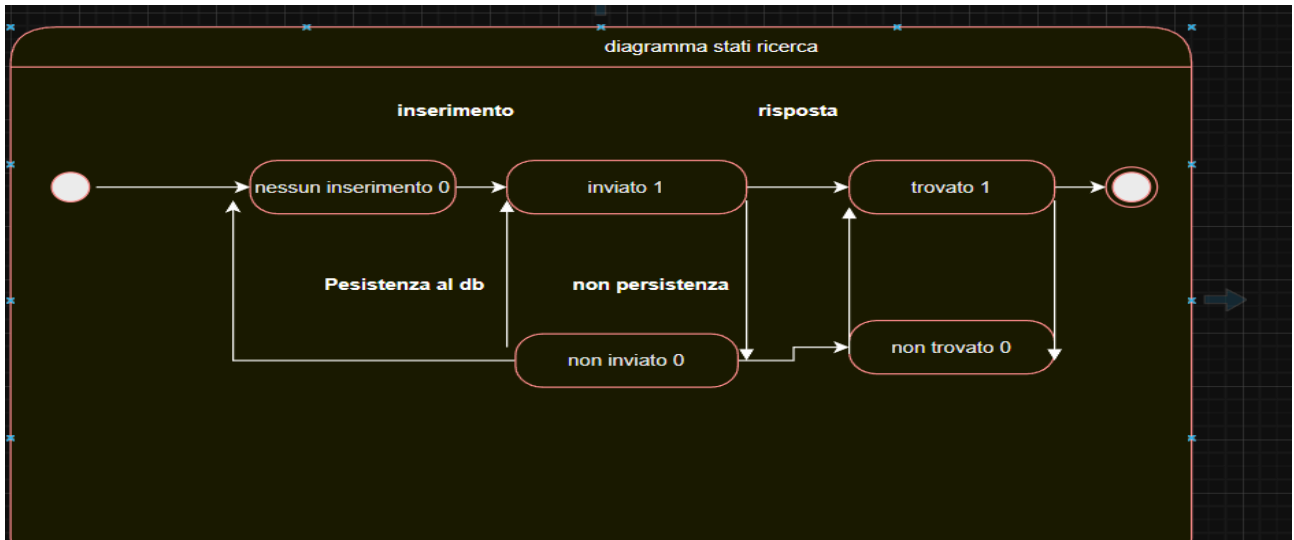
JavaBean POJO che memorizza i dati degli studenti.

Diagrammi

- Diagramma degli stati per l'inserimento



- Diagramma degli stati per la ricerca



Layer Data

vedi documentazione pdf allegato generato con phpmyadmin

Conclusione

L'implementazione si è conclusa senza criticità: la suddivisione in layer e l'uso di EJB @Stateless e DAO hanno garantito uno sviluppo stabile e privo di complicazioni.

Software utilizzati: Netbeans, draw.io,phpmyadmin design mysql,Papyrus,projectLibre